

阶段 02 实验报告：页表与内存管理

仓库地址在此处

此份报告请执行 `git branch vm` 来查看

1. 实验目标

通过深入分析 xv6 的内存管理系统，理解虚拟内存的工作原理，独立实现物理内存分配器和页表管理系统。最终能够正确分配/释放物理页、建立虚拟地址映射，并在 QEMU 中启用分页运行。

实验要求掌握以下内容：

1. RISC-V Sv39 虚拟内存机制；
2. 物理内存分配器 (free list) 的设计与实现；
3. 页表结构与遍历算法；
4. xv6 的 `walk()`、`mappages()` 等函数逻辑；
5. 在内核中启用分页的流程。

2. 任务完成情况与问题回答

2.1 任务 1: 深入理解 Sv39 页表机制

1. 虚拟地址分解

- Sv39 使用 39 位虚拟地址，划分为三级索引和页内偏移：

```
1 | [VPN[2]:9bit] [VPN[1]:9bit] [VPN[0]:9bit] [page  
offset:12bit]
```

Fence 1

- 每个 VPN (虚拟页号段) 对应 9 位，这是因为：
 - 页大小固定为 4KB (偏移 12 位)；
 - 页表项 (PTE) 大小为 8 字节，一个页表刚好存放 512 项 (2^9)；
 - 因此每级页表需要 9 位索引。
- 各级 VPN 的作用：逐级索引，最终找到物理页号 (PPN)。

2. 页表项 (PTE) 格式

- 关键位：
 - V: 有效位

- R/W/X: 读写执行权限
- U: 用户态可访问
- PPN 提取: `PTE >> 10` 得到物理页号, 再左移 12 位即可恢复物理地址。

3. 深入思考

- 为什么是 三级页表?
 - 三级足以覆盖 39 位虚拟地址空间 ($\approx 512\text{GB}$), 在 RISC-V 64 的通用设计中平衡了内存开销和寻址能力。
- 中间级页表项 R/W/X 应如何设置?
 - 通常清零, 只标记 $V=1$, 表示“继续索引”, 不直接用于访问。
- 如何理解“页表存储在物理内存中”?
 - 页表本质上就是一系列物理页, SATP 寄存器保存根页表的物理页号。

2.2 任务 2: 分析 xv6 的物理内存分配器

1. 核心数据结构

```
1 struct run {  
2     struct run *next;  
3 };
```

Fence 2

- 巧妙之处: 直接利用空闲页本身存放链表指针, 无需额外元数据。

2. kinit() 初始化

- 通过 `freerange(end, PHYSTOP)` 确定可分配范围;
- 将每个页按 4KB 对齐后加入 free list;
- 对齐原因: 保证分配出的都是完整页, 避免跨页。

3. kalloc()/kfree()

- `kalloc()` 从链表头取出一页, 时间复杂度 $O(1)$;
- `kfree()` 将一页重新挂回 free list;
- double-free 防护: 把已释放页填满垃圾值, 让系统访问悬空指针的时候快速崩溃。
- 优缺点:
 - 优点: 实现简单、速度快;
 - 缺点: 无统计功能, 不支持大块分配。

2.3 任务 3: 设计物理内存管理器

目前的物理内存管理器采用链式管理, 详见对应代码, 后续可扩展为伙伴算法分配。

2.4 任务 4: 理解 xv6 页表管理

1. walk() 遍历

- 从根页表开始, 逐级提取 VPN;
- 无效项时, 根据 `alloc` 参数决定是否新建中间页表。

2. mappages() 建立映射

- 地址必须页对齐;
- 调用 `kalloc` 分配物理页;
- 设置 PTE 权限位 (R/W/X/U) ;。

3. 地址转换相关宏

```
1 | #define PGROUNDUP(sz)  (((sz)+PGSIZE-1) & ~(PGSIZE-1))
2 | #define PGROUNDOWN(a) (((a)) & ~(PGSIZE-1))
3 | #define PTE_PA(pte)    (((pte) >> 10) << 12)
```

Fence 3

2.5 任务 5: 实现页表管理系统

• 接口设计

接口函数较多, 详见对应代码。

• 实现步骤

1. 从虚拟地址提取 VPN;
 2. 遍历/新建页表;
 3. 建立 PTE 并设置权限。
-

2.6 任务 6: 启用虚拟内存

1. kvminit()

- 创建内核页表;
- 恒等映射代码段、数据段、设备地址。

2. kvmithart()

- 设置 `satp`: MODE=8 (Sv39), PPN 指向根页表;

3. 思考题回答

1. 我的分配器 vs xv6

- 暂时都是链式内存分配器。

2. 内存安全

- 防止恶意利用：释放后用垃圾值覆盖内存页；
- 页表权限严格控制，用户态不能访问内核内存。

3. 性能瓶颈

- `free list` 不能快速分配大块内存；
- 优化可用 `buddy system` 或 `slab` 分配器。

4. 扩展性

- 支持用户进程：需要用户态页表和内核/用户地址空间隔离；
- 内存共享/写时复制：修改页表权限，结合缺页异常处理（未实现）。

5. 错误恢复

- 页表创建失败时，必须释放已分配的中间页表；
 - 如果页面分配失败，内核进入 `panic` 状态。
-

4. 额外任务实现：

4.1 物理页面分配检测

```

mem_pages = 0x00000000087a2d000, data = 16843009
mem_pages = 0x00000000087a2c000, data = 16843009
mem_pages = 0x00000000087a2b000, data = 16843009
mem_pages = 0x00000000087a2a000, data = 16843009
mem_pages = 0x00000000087a29000, data = 16843009
mem_pages = 0x00000000087a28000, data = 16843009
mem_pages = 0x00000000087a27000, data = 16843009
mem_pages = 0x00000000087a32000, data = 16843009
cpu 2 alloc over
mem_pages = 0x00000000087a26000, data = 16843009
mem_pages = 0x00000000087ffd000, data = 16843009
mem_pages = 0x00000000087fff000, data = 16843009
mem_pages = 0x00000000087fc8000, data = 16843009
mem_pages = 0x00000000087fee000, data = 16843009
mem_pages = 0x00000000087fba000, data = 16843009
mem_pages = 0x00000000087fa5000, data = 16843009
mem_pages = 0x00000000087f7d000, data = 16843009
mem_pages = 0x00000000087f79000, data = 16843009
mem_pages = 0x00000000087fa4000, data = 16843009
mem_pages = 0x00000000087f90000, data = 16843009
mem_pages = 0x00000000087f45000, data = 16843009
mem_pages = 0x00000000087f8c000, data = 16843009
mem_pages = 0x00000000087f42000, data = 16843009

```

Figure 1

此结果可以在 `git checkout 物理内存分配` 后复现。

4.2 虚拟内存映射检测

这里暂时将 `TRAMPOLINE` 的虚拟地址等映射到物理地址，第一次还未执行 `mappages`，`walk` 的结果显示是无效页表项；之后 `hart0`，`hart1`，`hart2` 均访问到正确的物理地址。

```

xv6 is booting!
[debug]: walk: va 0x0000003fffffff000
[debug]: walk: level 2, index 255, pte 0x0000000000000000 (new page table allocated at 0x0000000087fb9000)
[debug]: walk: level 1, index 511, pte 0x0000000000000000 (new page table allocated at 0x0000000087fb8000)
[debug]: walk: level 0, index 511, pte 0x0000000000000000
[debug]: the 0x0000003fffffff000 va is currently not mapped to a leaf (no PA)
[debug]: walk: va 0x0000003fffffff000
[debug]: walk: level 2, index 255, pte 0x0000000021fee401 (valid)
[debug]: walk: level 1, index 511, pte 0x0000000021fee001 (valid)
[debug]: walk: level 0, index 511, pte 0x0000000ffffffc0b
[debug]: the 0x0000003fffffff000 va is mapped to pa 0x0000003fffffff000

hart 1 starting!
[debug]: walk: va 0x0000003fffffff000
[debug]: walk: level 2, index 255, pte 0x0000000021fee401 (valid)
[debug]: walk: level 1, index 511, pte 0x0000000021fee001 (valid)
[debug]: walk: level 0, index 511, pte 0x0000000ffffffc0b
[debug]: the 0x0000003fffffff000 va is mapped to pa 0x0000003fffffff000

hart 2 starting!
[debug]: walk: va 0x0000003fffffff000
[debug]: walk: level 2, index 255, pte 0x0000000021fee401 (valid)
[debug]: walk: level 1, index 511, pte 0x0000000021fee001 (valid)
[debug]: walk: level 0, index 511, pte 0x0000000ffffffc0b
[debug]: the 0x0000003fffffff000 va is mapped to pa 0x0000003fffffff000

```

Figure 2

此结果可以在 [git checkout 页表功能](#) 后复现。

5. 总结

本实验通过实现 **物理内存分配器** 和 **页表管理系统**, 掌握了 RISC-V Sv39 分页机制及 xv6 的内存管理框架。实验的核心收获是理解:

- 虚拟地址的三级分解与 PTE 格式;
- free list 分配器的简单高效设计;
- 页表遍历和映射的实现逻辑;
- 启用分页的关键步骤与陷阱。

为后续的 **进程管理与用户态支持** 打下基础。